

TP 3: JDBC

Introduction :	1
I. configuration de la BDD.....	2
II. Création de la classe Livre.....	3
III. Manipulation des données.....	3
IV. Synchronisation.....	6
Conclusion.....	6

Introduction :

L'objectif de ce TP était d'apprendre à faire communiquer un programme Java avec une base de données relationnelle MySQL. J'ai utilisé le pilote JDBC pour établir la connexion, exécuter des requêtes SQL (SELECT, INSERT, UPDATE) et afficher les données dans une interface graphique Swing.

I. configuration de la BDD

Avant de commencer à coder, j'ai mis en place l'environnement de travail en démarrant le serveur MySQL et en intégrant le pilote JDBC au projet. J'ai ensuite créé une base de données nommée slam2_td2 ainsi qu'une table Livre.

ISBN	titre ▲ 1	auteur	prix
4293996815	Absalon Absalon !	William Faulkner	29.5
5979656845	Berlin Alexanderplatz	Alfred Doblin	30
6188391185	Cent ans de solitude	Gabriel Garcia Marquez	6
978-2-PRO-JAVA	Coder comme un pro en Java	M.Gravouil	99
1317442277	Contes	Hans Christian Andersen	10
6154636942	Crime et Chatiment	Fiodor Dostoievski	30
5879805162	Decameron	Boccace	16
4548950963	Divine Comedie	Dante Alighieri	25
5147239320	Don Quichotte	Miguel de Cervantes	15

II. Création de la classe Livre

J'ai développé une classe Livre qui ne se contente pas de stocker des données en mémoire : elle synchronise aussi les modifications directement dans la base de données.

Pour y parvenir, j'ai modifié les setters (mutateurs). Par exemple, la méthode setPrix accepte maintenant un objet Statement en paramètre, ce qui lui permet d'exécuter automatiquement une requête UPDATE dès qu'une valeur est modifiée.

```
// 4. SETTERS JDBC (Mutateurs avec mise à jour BDD)

//Set ISBN
public void setISBN(String isbn, Statement stmt) {
    this.isbn = isbn;
    try {

        String requete = "UPDATE Livre SET ISBN = '" + isbn + "' WHERE titre = '" + this.titre + "'";
        stmt.executeUpdate(requete);
        System.out.println(x: "Base de données mise à jour : Nouvel ISBN enregistré.");    Replace this
    } catch (SQLException e) {
        System.out.println("Erreur SQL : " + e.getMessage());    Define a constant instead of duplicat
    }
}

//Set Titre
public void setTitre(String titre, Statement stmt) {
    this.titre = titre;
    try {
        String requete = "UPDATE Livre SET titre = '" + titre + "' WHERE ISBN = '" + this.isbn + "'";
        stmt.executeUpdate(requete);
        System.out.println(x: "Base de données mise à jour : Nouveau titre enregistré.");    Replace th
    } catch (SQLException e) {
        System.out.println("Erreur SQL : " + e.getMessage());    Replace this use of System.out by a l
    }
}
```

III. Manipulation des données

Grâce à la modification des setters, toute mise à jour du prix d'un livre (par exemple, l'ISBN "1317442277" passé à 10€) est immédiatement répercutée dans MySQL, sans aucune intervention manuelle de ma part.

J'ai aussi ajouté le livre "Coder comme un pro en Java" de M. Gravouil pour tester le nouveau constructeur. Le livre est simultanément ajouté à la liste visuelle et inséré dans la table SQL.

```

/**
 * Constructeur "Intelligent"
 * Crée l'objet et l'insère directement dans la BDD via le Statement.
 */
public Livre(String isbn, String titre, String auteur, int prix, Statement stmt) {
    // A. Initialisation de l'objet Java
    this.isbn = isbn;
    this.titre = titre;
    this.auteur = auteur;
    this.prix = prix;

    // B. Insertion SQL directe
    try {
        String requete = "INSERT INTO Livre (ISBN, titre, Auteur, prix) VALUES ('"
            + isbn + "', '"
            + titre + "', '"
            + auteur + "', "
            + prix + ")";

        stmt.executeUpdate(requete);
        System.out.println("Succès : Le livre " + titre + " a été ajouté à la BDD !")
    } catch (SQLException e) {
        System.out.println("Erreur lors de l'insertion en BDD : " + e.getMessage());
    }
}

```

Dans le programme principal, j'ai testé deux opérations :

- Mise à jour (UPDATE) : j'ai modifié le prix du livre (ISBN 1317442277) pour le fixer à 10€.
- Insertion (INSERT) : j'ai utilisé un constructeur "intelligent" pour créer le livre "Coder comme un pro en Java" de M. Gravouil et l'insérer directement en base.

```

// B. Configuration de la connexion JDBC
String url = "jdbc:mysql://localhost:3306/slam2_td2";
String login = "root";
String passwd = ""; // Pas de mot de passe par défaut sur XAMPP

try {
    // 1. Établissement de la connexion
    Connection con = DriverManager.getConnection(url, login, passwd);    Add password protection to this database. [+1 location]
    Statement stmt = con.createStatement();

    // QUESTION 7 : MODIFICATION D'UN PRIX (UPDATE)

    Livre livreAmodifier = new Livre(isbn: "1317442277", titre: "Titre temporaire", auteur: "Auteur temporaire", prix: 0);

    // Appel de la méthode intelligente setPrix avec Statement
    livreAmodifier.setPrix(prix: 10, stmt);
    System.out.println(x: "-> Le prix a été modifié à 10€ pour l'ISBN 1317442277.");    Replace this use of System.out by a logger.

    // QUESTION 9 : AJOUT DU LIVRE M. GRAVOUIL (INSERT)

    int prixEntier = (int) 99.50;

    // Création + Insertion BDD via le constructeur "Intelligent" (créé à la question 8)
    Livre livrePro = new Livre(isbn: "978-2-PRO-JAVA", titre: "Coder comme un pro en Java", auteur: "M.Gravouil", prixEntier, stmt);

    // Ajout à la liste
    listeDesLivres.add(livrePro);
    System.out.println(x: "-> Livre ajouté avec succès à la liste et à la BDD.");    Replace this use of System.out by a logger.

    // 2. Fermeture
    stmt.close();
    con.close();

} catch (SQLException e) {
    e.printStackTrace();
}

return listeDesLivres;

```

```

-> Le prix a été modifié à 10€ pour l'ISBN 1317442277.
Succès : Le livre Coder comme un pro en Java a été ajouté à la BDD !
-> Livre ajouté avec succès à la liste et à la BDD.

```

--- CONTENU FINAL DE LA BIBLIOTHEQUE ---

Taille de la bibliotheque: 5

Livre: Titre 1 (11111) - 1.0€

Livre: Titre 2 (22222) - 2.0€

Livre: Titre 3 (33333) - 3.0€

Livre: Titre 4 (44444) - 4.0€

Livre: Coder comme un pro en Java (978-2-PRO-JAVA) - 99.0€

IV. Synchronisation

Dans la fenêtre principale, j'ai supprimé la création manuelle des livres pour la remplacer par un chargement dynamique depuis la base de données.

- Chargement au démarrage : au lancement de l'application, j'exécute un `SELECT * FROM Livre` pour récupérer tous les livres.
- Traitement des résultats : j'utilise une boucle `while(rs.next())` pour parcourir le `ResultSet` et reconstruire la liste de livres avec les vraies données de la base.

```
// --- INITIALISATION DONNÉES (BDD) ---
String url = "jdbc:mysql://localhost:3306/slam2_td2";
String login = "root";
String passwd="";

try {
    Class.forName(className: "com.mysql.cj.jdbc.Driver"); // Driver récent
    Connection con = DriverManager.getConnection(url, login, passwd); // Add password protection to this database. [+1 local
    Statement stmt = con.createStatement(); // Use try-with-resources or close this "Statement" in a "finally" clause.

    // ATTENTION : J'ai commenté cette ligne car elle fait planter le programme
    // si l'ISBN 999 existe déjà. C'était juste pour un test.
    // String sql = "INSERT INTO `livre` (`ISBN`, `TITRE`, `AUTEUR`, `PRIX`) VALUES ('999', 'test', 'Auteur test', 10); ";
    // stmt.executeUpdate(sql);

    // Lecture de la BDD
    String requete = "SELECT * FROM Livre"; // Don't use the query "SELECT *".
    ResultSet rs = stmt.executeQuery(requete);

    while (rs.next()) {
        String isbn = rs.getString(columnLabel: "ISBN");
        String titre = rs.getString(columnLabel: "titre");
        String auteur = rs.getString(columnLabel: "Auteur");

        int prix = rs.getInt(columnLabel: "prix");

        // On crée l'objet Java et on l'ajoute à la liste
        Livre livreBdd = new Livre(isbn, titre, auteur, prix);
        al.add(livreBdd);
    }

    rs.close();
    stmt.close();
    con.close();
} catch (Exception e) {
    e.printStackTrace();
    JOptionPane.showMessageDialog(parentComponent: null, "Erreur de connexion BDD : " + e.getMessage());
}
```

Conclusion

Ce TP m'a permis de comprendre le rôle essentiel du JDBC dans les applications Java. J'ai appris à distinguer les méthodes `executeQuery` (pour lire des données) et `executeUpdate` (pour les modifier ou en insérer). Mon application est désormais capable de conserver ses données entre deux exécutions, ce qui représente une étape clé dans le développement d'une application métier complète.